

IPF – Inpact Platform Foundation

A virtual-enterprise operations layer bridging fresh grads to real industry work: product research and spec generation → task creation → tutorial matching → job-seeker intake → real coding work → PR review. This document is a complete, current-state snapshot for whoever picks this up next.

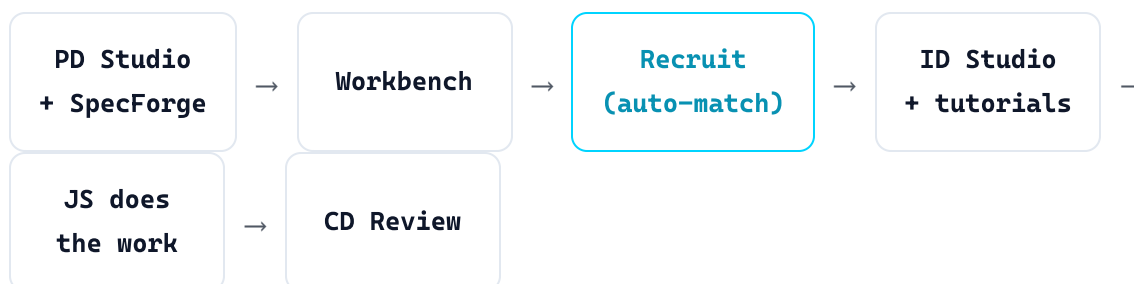
Repo: `D:\IPAAL` Remote: `github.com/phaniblend/IPAAL` Branch: `main`

⚠ nothing from this build is committed yet

§1 Orientation – read this first

What IPF is, in the founder's terms, and where the ground truth lives.

IPF is **not** the lesson content itself — that's the same app's older, separate half (React/TS/Vue/Angular curriculum, the "cinematic landing" at inpact.live). IPF is the **ops layer bolted onto it**: a pipeline that takes a product idea, breaks it into real assignable tasks, matches job-seekers (JS) to those tasks by trade and skill, gets them a tutorial if they need one, and routes their work through review — gated so a task is never assignable until it's wired to a tutorial or explicitly exempt.



The authoritative build log is [docs/IPF_DEVGUIDE.md](#) (802 lines) — every design decision, every bug found and fixed, every live-verification, in chronological order.

This document is a *current-state index* into it, not a replacement. Read §5a-1 through §5a-7 there for the full story behind anything summarized here.

That file's own top-level §0/§5/§5a headers are stale — they still say "nothing in this repo is access-controlled" and list DeepSeek/Gemini keys as blocked. Both are wrong as of today: auth is fully built (§7 below), and the API keys have been funded and working since 2026-08-04. Trust the dated §5a-N entries and this document over the top-of-file summary until someone does a refresh pass on it.

§2 Where it lives

Local path, git remote, and two things to fix before your first commit.

LOCAL PATH

D:\IPAAL

GIT REMOTE (ORIGIN)

github.com/phaniblend/IPAAL

BRANCH

main

also on origin: `live-react-ts-locked`,
`production`

RELATED REPO (SEPARATE!)

D:\inpact-assistance-mods\IPAAL-main\IPAAL-main

The core-lesson engine IPF's "Try a core lesson" flow launches into — its own dev server, port 5174.

Every file described in this document is currently uncommitted on `main`. `git status` shows it all as modified/untracked — nothing has been pushed or reviewed. Run `git status` and `git diff --stat` before assuming anything here is "live" on the remote. This includes all of Auth, Recruit auto-matching, the Workbench JS-scoped view, the enterprise-readiness gate, and every bugfix in §9.

`oauthcreds.json` **at the repo root is untracked but *not* gitignored.** It's a 2-line plaintext file holding the real Google OAuth Client ID and Secret pasted in during setup. `.env` is correctly gitignored — this file was missed. Add it to `.gitignore` (or

delete it once the values are safely in `.env`) before anyone runs `git add -A`, or rotate the credential in Google Cloud Console if it's already leaked.

§3 Architecture

The six-stage pipeline, and the system-of-record convention every module follows.

The pipeline, stage by stage

STAGE	SCREEN / ROUTE	DOES WHAT
PD	<code>/pd-studio</code>	Product idea in → SpecForge (DeepSeek) breaks it into a normalized domain model → task breakdown → publish, writing real OneDev issues.
Workbench	<code>/workbench</code>	The task board itself. Core roles see everything; JS accounts see only their own matched task(s) (built today, §7).
Recruit	<code>/apply</code> , <code>/matching-queue</code>	JS applies (Google-authenticated) → auto-matched to an open task by trade + skill level the instant they apply. Matching Queue is the human fallback for anyone not auto-matched.
ID	<code>/id-studio</code>	Auto-drafts (Gemini) or manually authors the tutorial a task needs before it's assignable; Module Library is the reuse cache.
JS	<code>/assist-me</code>	Does the actual work — 3-column in-app workspace or local-instructions mode, wired to either an auto-drafted module or a real core lesson.
CD	<code>/cd-review</code>	Thin queue over OneDev's real Pull Requests; a human still reviews every PR by hand.

System of record

There is no app database. **OneDev issues are the database.** Every module's state is a convention over an issue's title + a `Key: value`-per-line description, parsed with `parseKV` (`server/onedev-client.js`). E.g. an application is an issue titled

`Application: Jane Doe - QA` in the `cohort-applications` project; a match is `Matched: Jane Doe → #30 Create manual test plan`; a task waiting on a tutorial carries `NeedsTutorial: true` in its description. Grep for the marker name across `src/` and `server/` to find every place that reads or writes it.

No background jobs, anywhere

There is no cron, queue, or webhook receiver in this app. Every side effect (a Mattermost ping, an auto-match, a low-supply alert) fires as the *direct synchronous result of a human action* — someone applying, someone loading a screen. If you're tempted to add a "check every N minutes" feature, that's a real architectural addition, not a config flag.

§4 External services

Four Docker containers plus two paid APIs. None of the containers are defined in this repo.

No `docker-compose.yml` exists. These containers were created once by hand outside this repo and just get stopped/started with the host machine. Run `docker ps -a` at the start of any session — don't assume they're up, and don't `docker run` fresh ones without checking first (a fresh OneDev won't have the pre-existing projects this whole build assumes).

CONTAINER	IMAGE	PORT(S)	PURPOSE
<code>onedev-inpact</code>	OneDev	6610–6611	The system of record — every issue/project this whole app reads and writes.
<code>ipf-mattermost</code>	Mattermost	8065	Team Messaging — every notification anywhere posts to its Town Square via one incoming webhook.
<code>ipf-postgres</code>	Postgres	5432 (internal)	Mattermost's own datastore. Not touched directly by IPF code.
<code>ipf-redis</code>	Redis (pre-built)	6379	Redis instance for <code>ipf-redis</code> container

LDAP seed data (`ou=core,dc=inpact,dc=live`)

NAME	UID	COREROLE
Priya Sharma	<code>priya_sharma</code>	PD-core
Arjun Mehta	<code>arjun_mehta</code>	PMGT-core
Kavya Nair	<code>kavya_nair</code>	ID-core

Passwords are in `.env` -adjacent seed scripts / your own memory from setup — not reproduced here. Anonymous LDAP reads are disabled (a real admin bind is required even for the uid→DN lookup step).

Paid / external APIs

- **DeepSeek** — SpecForge (PD Studio's Stage 1/2/3) and the mentor chat. Funded and working since 2026-08-04.
- **Gemini** — ID Module's tutorial auto-drafting. Funded and working since 2026-08-04.
- **Google OAuth** — JS applicant sign-in. Registered app, client ID/secret in `.env` (see §2's warning about `oauthcreds.json`). Currently in Google's "Testing" publish status — only the test users added under Audience → Test users can sign in until it's published.

§5 Running it locally

Two Node processes, one dev server, four containers.

1 - START THE CONTAINERS (DOCKER DESKTOP MUST BE RUNNING)

```
docker ps -a
docker start ipf-ldap ipf-mattermost ipf-postgres onedev-inpact # whichever show "Exited"
```

2 - INSTALL + START THE AI/API SERVER (PORT 3000)

```
npm install
node server/index.js
```

No nodemon — restart this manually after *every* server-side change. This tripped up nearly every server edit this session; get in the habit of `netstat -ano | grep :3000` → `taskkill //PID <pid> //F` → relaunch.

3 - START THE VITE DEV SERVER (PORT 5173)

```
npm run dev
```

`vite.config.js` now sets `host: true` (fixed today — it was binding to `[::1]` only, so `127.0.0.1:5173` was dead while `localhost:5173` worked). A `vite.config.js` change needs a full server restart, not just HMR.

4 - OPEN IT

```
http://localhost:5173          # or http://127.0.0.1:5173 - both work now
```

Full-stack sanity check

```
curl -s -o /dev/null -w "AI server: %{http_code}\n" http://127.0.0.1:3000/api/auth/me
curl -s -o /dev/null -w "OneDev proxy: %{http_code}\n" http://localhost:5173/onedev-api/issue
curl -s -o /dev/null -w "Vite: %{http_code}\n" http://localhost:5173/
```

§6 Directory map

IPF's slice of the repo — the lesson-engine half (`src/engines`, `src/ai-lessons`, etc.) is the older, separate system and out of scope here.

server/

- `index.js` - mounts every router, serves the SPA in prod
- `onedev-client.js` - the ONE OneDev client (`listIssues/createIssue/parseKV/...`)
- `auth-router.js`, `auth-session.js`, `ldap-auth.js`, `google-oauth.js`, `role-grants.js`
- `recruit-router.js` - auto-matching + low-supply alerts
- `specforge-router.js`, `id-router.js`, `assistance-sessions.js`, `assist-me-router.js`

notify-server.js – server-side Mattermost (no Vite proxy to lean on)

src/

auth/ – useAuth.js, RequireRole.jsx, CoreLogin.jsx
pd-studio/, specforge/ – Stage 1/2/3 generation UI + schemas
workbench/ – the task board (core view + JS-scoped view)
cohort-matching/ – Apply.jsx, MatchingQueue.jsx, matching.js (shared predicate)
instructional-design/ – ID Studio, Module Library, Draft Review
id-module/ – core-lesson matching, module generation
assist-me/, engines/assist/ – the JS workspace + generated tutorial modules
cd-review/, core-studio/, huddle-calendar/, contribution-monitor/, human-capital-reports/, cohorts/, team-messaging/
EnterpriseReadinessGate.jsx – the "before you apply" interstitial off the cinematic landing

docs/

IPF_DEVGUIDE.md – the full build log, read this next

§7 Auth system

Two identity paths, one session shape. Built this session — fully live-verified.

PATH	WHO	HOW
<code>POST /api/auth/core-login</code>	<code>-core</code> employees	LDAP bind (username/password against <code>ipf-ldap</code>) — never compares hashes, binds as the user.
<code>GET /api/auth/google/start</code>	JS applicants	Real Google OAuth. CSRF <code>state</code> is a server-memory one-time token (not a cookie — see the callout below).

Session: a signed JWT (`AUTH_SESSION_SECRET`) in an httpOnly, `sameSite:"lax"` cookie, 12h TTL. Shape:

```
{ sub, name, email, accountType: "core" | "js", coreRole?: string, roles: string[] }
```

`requireSession` / `requireRole( ... roles)` middleware in `server/auth-session.js` guard API routes; `<RequireRole roles={[ ... ]}>` in `src/main.jsx` guards frontend routes the same way. A `-core` account's `coreRole` (e.g. `"PD-`

guards mention routes and some may not (e.g. `core`) also satisfies the bare role name (`"PD"`) — a core employee already *is* their role; a JS has to be explicitly granted one via `role-grants.js`.

Real bug found and fixed today, via an actual Google sign-in attempt (not curl):

`/google/start` is reached through the Vite-proxied browser origin (`localhost:5173`), so a cookie set there scopes to host `localhost` — but Google's redirect lands directly on `127.0.0.1:3000`, a different host as far as the browser's cookie jar is concerned. The state cookie never arrived at the callback, failing every real sign-in. Fixed by moving CSRF state out of a cookie entirely into a server-memory one-time token that round-trips through Google as a URL param instead (`pendingOAuthStates` in `auth-router.js`).

Route guard map (`src/main.jsx`)

ROUTE	REQUIRES
<code>/pd-studio</code>	PD
<code>/id-studio</code>	ID
<code>/cd-review</code>	CD
<code>/workbench</code> , <code>/assist-me</code>	any signed-in session (core or JS)
<code>/matching-queue</code> , <code>/core-studio</code> , <code>/cohorts</code> , etc.	any core role
<code>/apply</code>	unguarded — frictionless applicant funnel, by design

§8 Built & verified

Condensed from `IPF_DEVGUIDE.md §5a-1` through `§5a-7`. Every item below was confirmed with a real click-through or a real API call, not just read as code.

- ✓ `live-verified` PD Studio: full spec → Stage 3 → publish flow, real OneDev writes.

- ✓ **live-verified** Skill-matched recruiting: two-tier trade/level gate, JS-tier default → TS/advanced unlock.
 - ✓ **live-verified** Recruit auto-matching: applying *is* the match trigger now, not a manual Matching Queue click. Matching Queue is the fallback for anything that didn't auto-match.
 - ✓ **live-verified** Google OAuth end-to-end (real account, not a test session) — sign-in, application, auto-match, "You're matched" screen.
 - ✓ **live-verified** LDAP **-core** login, both the grant and deny paths, through the real rendered UI.
 - ✓ **live-verified** Apply page: no free-text email field — Google sign-in *is* the last step of submitting, draft survives the OAuth round-trip via `sessionStorage`, auto-submits on return.
 - ✓ **live-verified** Workbench JS-scoped view — a JS sees only their own matched task(s), not the full admin board.
 - ✓ **live-verified** Low-supply alerts — PD-core gets pinged on Mattermost when a trade drops to ≤ 3 assignable tasks, once per 24h.
 - ✓ **live-verified** Enterprise-readiness gate on the cinematic landing (`inpect.live`'s "Start doing" CTA) — forks to Apply vs. plain lessons.
 - ✓ **live-verified** Core-lesson session/completion loop (ID Studio ↔ the separate IAAL-main lesson app) — full token/session round trip.
 - ✓ **live-verified** CD Review: thin queue over real OneDev Pull Requests, one-time Mattermost announce per PR.
-

§9 Open gaps

Real, known, not hedging. Ordered roughly by how likely they are to bite first.

- **gap** **A task can be matched to more than one applicant.** `bestTaskMatch` in `matching.js` never excludes a task that already has a `Matched:` issue. Found while building the low-supply alert; not yet fixed.
- **gap** **No backend tutorial runtime exists.** Every lesson (including anything Gemini auto-drafts) is a React component rendered live in-browser — there's no

sandboxed way to run/grade real backend code. Founder call: stay UI-only for now, expand later with Gemini auto-drafting BE content into a not-yet-designed backend lesson schema. This is a real infra project, not a content gap.

- **gap** **The legal/ownership disclaimer on the "You're matched" screen is passive text**, not an affirmative checkbox at apply time. Weaker than a real consent record if it ever needs to hold up.
- **gap** **"View your task in Workbench" can dead-end at a login wall** for an applicant who applied but has no session context carried through in every entry path — worth a UX pass.
- **gap** **No re-match sweep** when a blocked task later gets a tutorial and unblocks — Matching Queue is the manual safety net, no automatic catch-up (consistent with "no background jobs," but worth a human-action-triggered sweep like CD Review's own pattern).
- **policy** `/publish` trusts a client-supplied `deliveryProjectId` beyond the reserved-id check — fine for one trusted internal team, worth hardening if this is ever exposed more broadly.
- **docs debt** `IPF_DEVGUIDE.md`'s top-level §0/§5/§5a/§6 need a refresh pass — see the callout in §1 above.

§10 Environment variables

`.env` is gitignored and not reproduced here — names and purpose only.

VARIABLE	PURPOSE
<code>ONEDEV_API_USER</code> / <code>_PASS</code>	OneDev basic auth — the whole system of record.
<code>DEEPSEEK_API_KEY</code>	SpecForge generation + mentor chat.
<code>GEMINI_API_KEY</code> / <code>GEMINI_MODEL</code>	ID Module tutorial auto-drafting.
<code>AUTH_SESSION_SECRET</code>	Signs the session JWT — rotate before any real deployment, it's a dev placeholder.
<code>LDAP_URL</code> , <code>LDAP_BASE_DN</code> , <code>LDAP_CORE_OU</code> , <code>LDAP_ADMIN_DN</code> , <code>LDAP_ADMIN_PASSWORD</code>	Points at <code>ipf-ldap</code> for <code>-core</code> login.

<code>GOOGLE_OAUTH_CLIENT_ID</code> / <code>_SECRET</code> / <code>_REDIRECT_URI</code>	Google Cloud Console → APIs & Services → Credentials, project <code>INPACTPF</code> .
<code>IPF_FRONTEND_URL</code>	Where the OAuth callback sends the browser back to. Must be <code>http://localhost:5173</code> in this environment (see §7's bug writeup).
<code>LESSONS_BASE_URL</code>	The separate IAAL-main lesson app, port 5174.
<code>VITE_MATTERMOST_WEBHOOK_ID</code>	The one incoming webhook every notification posts to.
<code>VITE_SUPABASE_*</code> , <code>VITE_FIREBASE_*</code>	Pre-existing lesson-app auth — not part of IPF, empty is fine unless touching that half.

§11 Conventions worth keeping

The discipline that found most of the real bugs listed above.

- **Verify with a real call, not a code read.** Every OneDev/LDAP/Google API assumption in this codebase was confirmed against real source or a real live call before being built on — guessed API shapes were wrong more than once (see `createIssue`'s bare-number return in §9's history, or the JAX-RS bare-String body on `POST /issues/{id}/description`).
- **A same-origin hash navigation isn't a real remount.** Testing tools that just flip `location.hash` on an already-loaded SPA don't force React to remount the way a genuine fresh page load does — call `location.reload()` to trust a test.
- `npmx eslint <file>` **after every change**, and check pre-existing failures with `git stash` before treating a lint error as your own regression — `server/*.js` has a known, pre-existing gap where `process` / `Buffer` aren't declared as globals in the eslint config.
- **Restart the AI server after every server-side change** — no nodemon. `vite.config.js` changes need a full Vite restart too, not just HMR.
- **One normalization point beats N copies of a defensive guess.** `onedev-client.js`'s `createIssue` / `createProject` now normalize OneDev's inconsistent response shape once, at the source — three call sites had each

inconsistent response shape once, at the source — three call sites had each independently discovered and guarded against it, a fourth hadn't and silently corrupted real data until found live.
